Messages must be serialized and deserialized from byte arrays upon sending and receiving. This functionality is included in the following examples, but it can be implemented separately for better code modularity. See the **sample app**) for an example of this.

C#

```
public async void StartReceivingMessages() {

    // Initialize. The channel name must be known by all participant devices
    // that will communicate over it.
    RemoteSystemSessionMessageChannel messageChannel = new
RemoteSystemSessionMessageChannel(currentSession,
        "Everyone in Bob's Minecraft game",
        RemoteSystemSessionMessageChannelReliability.Reliable);

    // write the handler for incoming messages on this channel
    messageChannel.ValueSetReceived += async (sender, args) => {

        // Update UI: a message was received from the participant
args.Sender

        // Deserialize the message
        // (this app must know what key to use and what object type the
value is expected to be)
        ValueSet receivedMessage = args.Message;
        object rawData = receivedMessage["appKey"];
        object value = new ExpectedType(); // this must be whatever type is
expected

        using (var stream = new MemoryStream((byte[])rawData)) {
            value = new
DataContractJsonSerializer(value.GetType()).ReadObject(stream);
        }

        // do something with the "value" object
        //...
    };
}
```

Send messages

When the channel is established, sending a message to all of the session participants is straightforward.

C#

```
public async void
SendMessageToAllParticipantsAsync(RemoteSystemSessionMessageChannel
```

```
messageChannel, object value){

    // define a ValueSet message to send
    ValueSet message = new ValueSet();

    // serialize the "value" object to send
    using (var stream = new MemoryStream()){
        new DataContractJsonSerializer(value.GetType()).WriteObject(stream,
value);
        byte[] rawData = stream.ToArray();
            message["appKey"] = rawData;
    }

    // Send message to all participants. Ordering is not guaranteed.
    await messageChannel.BroadcastValueSetAsync(message);
}
```

In order to send a message to only certain participant(s), you must first initiate a discovery process to acquire references to the remote systems participating in the session. This is similar to the process of discovering remote systems outside of a session. Use a **RemoteSystemSessionParticipantWatcher** instance to find a session's participant devices.

C#

```
public void WatchForParticipants() {
    // "currentSession" is a reference to a RemoteSystemSession.
    RemoteSystemSessionParticipantWatcher watcher =
currentSession.CreateParticipantWatcher();

    watcher.Added += (sender, participant) => {
        // save a reference to "participant"
        // optionally update UI
    };

    watcher.Removed += (sender, participant) => {
        // remove reference to "participant"
        // optionally update UI
    };

    watcher.EnumerationCompleted += (sender, args) => {
        // Apps can delay data model render up until this point if they
wish.
    };

    // Begin watching for session participants
    watcher.Start();
}
```

When a list of references to session participants is obtained you can send a message to any set of them.

To send a message to a single participant (ideally selected on-screen by the user), simply pass the reference into a method like the following.

C#

```
public async void
SendMessageToParticipantAsync(RemoteSystemSessionMessageChannel
messageChannel, RemoteSystemSessionParticipant participant, object value) {

    // define a ValueSet message to send
    ValueSet message = new ValueSet();

    // serialize the "value" object to send
    using (var stream = new MemoryStream()){
        new DataContractJsonSerializer(value.GetType()).WriteObject(stream,
value);
        byte[] rawData = stream.ToArray();
            message["appKey"] = rawData;
    }

    // Send message to the participant
    await messageChannel.SendValueSetAsync(message,participant);
}
```

To send a message to multiple participants (ideally selected on-screen by the user), add them to a list object and pass the list into a method like the following.

C#

```
public async void SendMessageToListAsync(RemoteSystemSessionMessageChannel
messageChannel, IReadOnlyList<RemoteSystemSessionParticipant> myTeam, object
value){

    // define a ValueSet message to send
    ValueSet message = new ValueSet();

    // serialize the "value" object to send
    using (var stream = new MemoryStream()){
        new DataContractJsonSerializer(value.GetType()).WriteObject(stream,
value);
        byte[] rawData = stream.ToArray();
            message["appKey"] = rawData;
    }

    // Send message to specific participants. Ordering is not guaranteed.
    await messageChannel.SendValueSetToParticipantsAsync(message, myTeam);
}
```

Related topics

- Connected apps and devices (Project Rome)
- Remote Systems API reference

Continue user activity, even across devices

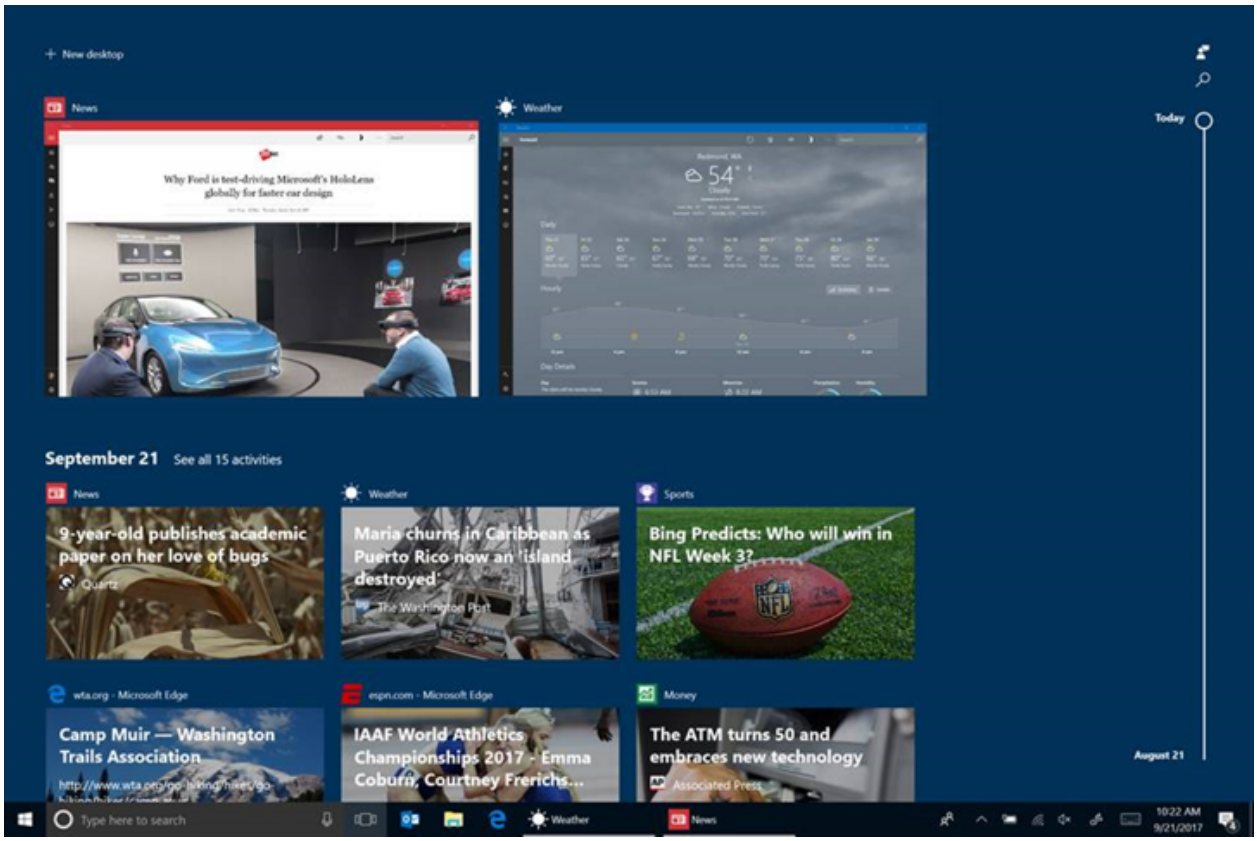
Article • 10/27/2022

This topic describes how to help users resume what they were doing in your app on their PC, and across devices.

> **Note**
>
> Starting in July 2021, users who have activity history synced across their Windows devices through their Microsoft Account (MSA) will no longer have the option to upload new activity in Timeline. They'll still be able to use Timeline and see their activity history (information about recent apps, websites and files) on their local PC. AAD-connected accounts won't be impacted.

User Activities and Timeline

Our time each day is spread across multiple devices. We might use our phone while on the bus, a PC during the day, then a phone or tablet in the evening. Starting in Windows 10 Build 1803 or later, creating a User Activity makes that activity appear in Windows Timeline and in Cortana's Pick up where I left off feature. Timeline is a rich task view that takes advantage of User Activities to show a chronological view of what you've been working on. It can also include what you were working on across devices.



Likewise, linking your phone to your Windows PC allows you to continue what you were doing previously on your iOS or Android device.

Think of a **UserActivity** as something specific the user was working on within your app. For example, if you are using a RSS reader, a **UserActivity** could be the feed that you are reading. If you are playing a game, the **UserActivity** could be the level that you are playing. If you are listening to a music app, the **UserActivity** could be the playlist that you are listening to. If you are working on a document, the **UserActivity** could be where you left off working on it, and so on. In short, a **UserActivity** represents a destination within your app so that enables the user to resume what they were doing.

When you engage with a **UserActivity** by calling UserActivity.CreateSession, the system creates a history record indicating the start and end time for that **UserActivity**. As you re-engage with that **UserActivity** over time, multiple history records are recorded for it.

Add User Activities to your app

A UserActivity is the unit of user engagement in Windows. It has three parts: a URI used to activate the app the activity belongs to, visuals, and metadata that describes the activity.

1. The ActivationUri is used to resume the application with a specific context. Typically, this link takes the form of protocol handler for a scheme (for example,

"my-app://page2?action=edit") or of an AppUriHandler (for example, `http://contoso.com/page2?action=edit`).
2. VisualElements exposes a class that allows the user to visually identify an activity with a title, description, or Adaptive Card elements.
3. Finally, Content is where you can store metadata for the activity that can be used to group and retrieve activities under a specific context. Often, this takes the form of https://schema.org data.

To add a **UserActivity** to your app:

1. Generate **UserActivity** objects when your user's context changes within the app (such as page navigation, new game level, etc.)
2. Populate **UserActivity** objects with the minimum set of required fields: ActivityId, ActivationUri, and UserActivity.VisualElements.DisplayText.
3. Add a custom scheme handler to your app so it can be re-activated by a **UserActivity**.

A **UserActivity** can be integrated into an app with just a few lines of code. For example, imagine this code in MainPage.xaml.cs, inside the MainPage class (note: assumes `using Windows.ApplicationModel.UserActivities;`):

C#

```
UserActivitySession _currentActivity;
private async Task GenerateActivityAsync()
{
    // Get the default UserActivityChannel and query it for our
UserActivity. If the activity doesn't exist, one is created.
    UserActivityChannel channel = UserActivityChannel.GetDefault();
    UserActivity userActivity = await
channel.GetOrCreateUserActivityAsync("MainPage");

    // Populate required properties
    userActivity.VisualElements.DisplayText = "Hello Activities";
    userActivity.ActivationUri = new Uri("my-app://page2?action=edit");

    //Save
    await userActivity.SaveAsync(); //save the new metadata

    // Dispose of any current UserActivitySession, and create a new one.
    _currentActivity?.Dispose();
    _currentActivity = userActivity.CreateSession();
}
```

The first line in the `GenerateActivityAsync()` method above gets a user's UserActivityChannel. This is the feed that this app's activities will be published to. The next line queries the channel of an activity called `MainPage`.

- Your app should name activities in such a way that same ID is generated each time the user is in a particular location in the app. For example, if your app is page-based, use an identifier for the page; if its document based, use the name of the doc (or a hash of the name).
- If there is an existing activity in the feed with the same ID, that activity will be returned from the channel with `UserActivity.State` set to Published). If there is no activity with that name, and new activity is returned with `UserActivity.State` set to **New**.
- Activities are scoped to your app. You do not need to worry about your activity ID colliding with IDs in other apps.

After getting or creating the **UserActivity**, specify the other two required fields: `UserActivity.VisualElements.DisplayText` and `UserActivity.ActivationUri`.

Next, save the **UserActivity** metadata by calling SaveAsync, and finally CreateSession, which returns a UserActivitySession. The **UserActivitySession** is the object that we can use to manage when the user is actually engaged with the **UserActivity**. For example, we should call `Dispose()` on the **UserActivitySession** when the user leaves the page. In the example above, we also call `Dispose()` on `_currentActivity` before calling `CreateSession()`. This is because we made `_currentActivity` a member field of our page, and we want to stop any existing activity before we start the new one (note: the `?` is the null-conditional operator which tests for null before performing the member access).

Since, in this case, the `ActivationUri` is a custom scheme, we also need to register the protocol in the application manifest. This is done in the Package.appmanifest XML file, or by using the designer.

To make the change with the designer, double-click the Package.appmanifest file in your project to launch the designer, select the **Declarations** tab and add a **Protocol** definition. The only property that needs to be filled out, for now, is **Name**. It should match the URI we specified above, `my-app`.

Now we need to write some code to tell the app what to do when it's been activated by a protocol. We'll override the `OnActivated` method in App.xaml.cs to pass the URI on to the main page, like so:

C#

```
protected override void OnActivated(IActivatedEventArgs e)
{
    if (e.Kind == ActivationKind.Protocol)
    {
```

```
        var uriArgs = e as ProtocolActivatedEventArgs;
        if (uriArgs != null)
        {
            if (uriArgs.Uri.Host == "page2")
            {
                // Navigate to the 2nd page of the  app
            }
        }
    }
    Window.Current.Activate();
}
```

What this code does is detect whether the app was activated via a protocol. If it was, then it looks to see what the app should do to resume the task it is being activated for. Being a simple app, the only activity this app resumes is putting you on the secondary page when the app comes up.

Use Adaptive Cards to improve the Timeline experience

User Activities appear in Cortana and Timeline. When activities appear in Timeline, we display them using the Adaptive Card framework. If you do not provide an adaptive card for each activity, Timeline will automatically create a simple activity card based on your application name and icon, the title field and optional description field. Below is an example Adaptive Card payload and the card it produces.

]

Example adaptive card payload JSON string:

JSON

```
{
  "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
```

```
  "type": "AdaptiveCard",
  "version": "1.0",
  "backgroundImage":
"https://winblogs.azureedge.net/win/2017/11/eb5d872c743f8f54b957ff3f5ef3066b
.jpg",
  "body": [
    {
      "type": "Container",
      "items": [
        {
          "type": "TextBlock",
          "text": "Windows Blog",
          "weight": "bolder",
          "size": "large",
          "wrap": true,
          "maxLines": 3
        },
        {
          "type": "TextBlock",
          "text": "Training Haiti’s radiologists: St. Louis doctor takes her
teaching global",
          "size": "default",
          "wrap": true,
          "maxLines": 3
        }
      ]
    }
  ]
}
```

Add the Adaptive Cards payload as a JSON string to the **UserActivity** like this:

C#

```
activity.VisualElements.Content =
Windows.UI.Shell.AdaptiveCardBuilder.CreateAdaptiveCardFromJson(jsonCardText
); // where jsonCardText is a JSON string that represents the card
```

Cross-platform and Service-to-service integration

If your app runs cross-platform (for example on Android and iOS), or maintains user state in the cloud, you can publish UserActivities via Microsoft Graph. Once your application or service is authenticated with a Microsoft Account, it just takes two simple REST calls to generate Activity and History objects, using the same data as described above.

Summary

You can use the UserActivity API to make your app appear in Timeline and Cortana.

- Learn more about the **UserActivity** API
- Check out the sample code.
- See more sophisticated Adaptive Cards.
- Publish a **UserActivity** from iOS, Android or your web service via Microsoft Graph.
- Learn more about Project Rome on GitHub.

Key APIs

- UserActivities namespace

Related topics

- User Activities (Project Rome docs)
- Adaptive cards
- Adaptive cards visualizer, samples
- Handle URI activation
- Engaging with your customers on any platform using the Microsoft Graph, Activity Feed, and Adaptive Cards
- Microsoft Graph

User Activities best practices

Article • 10/20/2022

This guide outlines the recommended practices for creating and updating User Activities. For an overview of the User Activities feature on Windows, see Continue user activity, even across devices. Or, see the User Activities section of Project Rome for the implementations of Activities on other development platforms.

> **Note**
>
> Starting in July 2021, users who have activity history synced across their Windows devices through their Microsoft Account (MSA) will no longer have the option to upload new activity in Timeline. They'll still be able to use Timeline and see their activity history (information about recent apps, websites and files) on their local PC. AAD-connected accounts won't be impacted.

When to create or update User Activities

Because every app is different, it's up to each developer to determine the best way to map actions within the app to User Activities. Your User Activities will be showcased in Cortana and Timeline, which are focused on increasing users' productivity and efficiency by helping them get back to content they visited in the past.

General guidelines

- **Record a single activity for a group of related user actions.** This is especially relevant for music playlists or TV Shows: a single Activity can be updated at regular intervals to reflect the user's progress. In this case, you will have a single User Activity with multiple History Items representing periods of engagement across multiple days or weeks. The same applies to document-based activities on which the user makes gradual progress within your app.
- **Store user data in the cloud.** If you want to support cross-device Activities, you'll need to make sure the content required to re-engage this Activity is stored to a cloud location. Device-specific Activities will appear on Timeline on the device where the activity was created but may not appear on other devices.
- **Do not create Activities for actions that users will not need to resume.** If your application is used to complete simple, one-time operations that do not persist status, you probably do not need to create a User Activity.

- **Do not create Activities for actions completed by other users.** If an external account sends the user a message or @-mentions them within your app, you should not create an Activity for this. This type of action is better served by Action Center Notifications.
    - Collaboration scenarios are an exception: If multiple users are working on the same activity together (such as a Word document), there will be cases in which another user has made changes after your user. In this case, you may want to update the existing Activity to reflect changes that were made to the document. This would involve updating the existing User Activity content data without creating a new History Item.

Guidelines for specific types of apps

While every app is different, most apps will fall into one of the following interaction patterns.

- **Document-based apps** — Create one Activity per document, with one or more History Items reflecting periods of use. It is important to update your Activity as changes are made to the document.
- **Games** — Create one Activity for each game save or world. If your game supports only a single sequence of levels, you can re-publish the same Activity over time, although you may wish to update the content data to show the latest progress or achievements.
- **Utility apps** — If there is nothing within your app that users would need to leave and resume, you do not need to use User Activities. A good example is a simple app like Calculator.
- **Line-of-business apps** — Many apps exist for managing simple tasks or workflows. Create one activity for each separate workflow accessed through your app (for example, expense reports would each be a separate Activity, so that the user could then click an Activity to see if a particular report was approved).
- **Media playback apps** — Create one Activity per logical grouping of content (such as a playlist, program, or standalone content). The underlying question for app developers is whether a each piece of content (TV episode, song) counts as standalone content or part of a collection. As a general rule, if the user opts to play a collection or sequential content, the collection as a whole is the activity. If they opt to play a single piece of content, then that one piece of content is the activity. See more specific guidelines below.
    - **Music: Album/Artist/Genre** — If the user selects an Album, Artist, or Genre and hits **play**, that collection is the activity; do not write a separate Activity for each song. For short collections like a single album or collections being played back in a random order, you may not need to update the Activity to reflect the user's

current position. For long sequential playback such as an album or playlist, recording your position within the album might make sense.
    - **Music: smart playlists** — Applications which play music in a random order should record a single Activity for that playlist. If the user plays the playlist a second time, you would create additional history records for the same Activity. Recording the user's current position in the playlist is not necessary because the ordering is random.
    - **TV series** — If your app is configured to play the next episode after the current one is complete, you should write a single Activity for the TV series. As you play the various episodes across multiple viewing sessions, you'll update your Activity to reflect the current position in the series, and multiple history records will be created.
    - **Movie** — A movie is a single piece of content and should have its own history record. If the user stops watching the movie part-way through, it is desirable to record their position. When they wish to resume it in the future, the Activity could resume the movie where they left off, or even ask the user if they wish to resume or start at the beginning.

User Activity design

User Activities consist of three components: an activation URI, visual data, and content metadata.

- The activation URI is a URI that can be passed to an application or experience in order to resume the application with a specific context. Typically, these links take the form of protocol handler for a scheme (for example, "my-app://page2?action=edit"). It is up to the developer to determine how URI parameters will be handled by their app. See Handle URI activation for more information.
- The visual data, consisting of a set of required and optional properties (for example: title, description, or Adaptive Card elements), allow users to visually identify an Activity. See below for guidelines on creating Adaptive Card visuals for your Activity.
- The content metadata is JSON data that can be used to group and retrieve activities under a specific context. Typically, this takes the form of http://schema.org data. See below for guidelines on filling out this data.

Adaptive Card design guidelines

When Activities appear in Timeline, they are displayed using the Adaptive Card framework. If the developer does not provide an Adaptive Card for each Activity,

Timeline will automatically create a simple card based on the app name/icon, the required Title field, and the optional Description field.

App developers are encouraged to provide custom cards using the simple Adaptive Card JSON schema. See the Adaptive Cards documentation for technical instructions on how to construct Adaptive Card objects. Refer to the guidelines below for designing Adaptive Cards in User Activities.

- Use images
  - Use a unique image for each Activity, if possible. Your application name and icon will automatically be displayed next to your Activity's card; additional images will help users locate the Activity they are looking for.
  - Images should not include text that the user is expected to read. This text won't be available to users with accessibility needs and cannot be searched.
  - If the image doesn't contain text and can be cropped to about a 2:1 ratio, you should use it as a background image. This results in a bold activity card which will stand out in Timeline. The image will be darkened slightly to ensure the text remains visible on the card, and you are encouraged to only use the Activity Name in this case, as smaller text can become hard to read.
  - If the image cannot be cropped to 2:1, you should put it within the Activity Card.
    - If the aspect ratio is Square or Portrait, anchor the image on the right side of the card with no margins.
    - If the aspect ratio is Landscape, anchor the image to the upper-right corner of the card.
- Each activity is required to provide an Activity Name, which should always be shown.
  - This name should be displayed in the upper-left corner of the card using the large bold text option. It is important that the name is easily recognizable, as this is the only part users will see when the activity is shown in Cortana scenarios. Showing the same name in Timeline makes it easier for users to browse a large number of Activities.
- Use the same visual style for all of the Activities from your app, so that users can easily locate your app's activities in the Timeline.
  - For example, Activities should all use the same background color.
- Use supplemental text information sparingly.
  - Avoid filling the card with text, and only use supplemental information that aids users in finding the right activity or reflects state information (such as the current progress in a particular task).

Content metadata guidelines

User Activities can also contain content metadata, which Windows and Cortana use to categorize Activities and generate inferences. Activities can then be grouped around a particular topic, such as a location (if the user is researching vacations), object (if the user is researching something) or action (if the user is shopping for a particular product across different apps and websites). It's a good idea to represent both the nouns and the verbs involved in an activity.

In the following example, the content metadata JSON, following the standards of Schema.org, represents the scenario: "John played Angry Birds with Steve."

JSON

```
// John played angry birds with Steve.
{
  "@context": "http://schema.org",
  "@type": "PlayAction",
  "agent": {
    "@type": "Person",
    "name": "John"
  },
  "object": {
    "@type": "MobileApplication",
    "name": "Angry Birds."
  },
  "participant": {
    "@type": "Person",
    "name": "Steve"
  }
}
```

Key APIs

- UserActivities namespace

Related topics

- User Activities (Project Rome docs)
- Adaptive cards
- Adaptive cards visualizer, samples
- Handle URI activation
- Microsoft Graph

C++/WinRT

Article • 11/15/2023

C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API. With C++/WinRT, you can author and consume Windows Runtime APIs using any standards-compliant C++17 compiler. The Windows SDK includes C++/WinRT; it was introduced in version 10.0.17134.0 (Windows 10, version 1803).

C++/WinRT is for any developer interested in writing beautiful and fast code for Windows. Here's why.

The case for C++/WinRT

https://www.youtube-nocookie.com/embed/rJxQnhiK4TQ

The C++ programming language is used both in the enterprise and independent software vendor (ISV) segments for applications where high levels of correctness, quality, and performance are valued. For example: systems programming; resource-constrained embedded and mobile systems; games and graphics; device drivers; and industrial, scientific, and medical applications, to name but some.

From a language point of view, C++ has always been about authoring and consuming abstractions that are both type-rich and lightweight. But the language has changed radically since the raw pointers, raw loops, and painstaking memory allocation and releasing of C++98. Modern C++ (from C++11 onward) is about clear expression of ideas, simplicity, readability, and a much lower likelihood of introducing bugs.

For authoring and consuming Windows APIs using C++, there is C++/WinRT. This is Microsoft's recommended replacement for the C++/CX language projection, and the Windows Runtime C++ Template Library (WRL).

You use standard C++ data types, algorithms, and keywords when you use C++/WinRT. The projection does have its own custom data types, but in most cases you don't need to learn them because they provide appropriate conversions to and from standard types. That way, you can continue to use the standard C++ language features that you're accustomed to using, and the source code that you already have. C++/WinRT makes it extremely easy to call Windows APIs in any C++ application, from Win32 to the Windows AppSDK to UWP.

C++/WinRT performs better and produces smaller binaries than any other language option for the Windows Runtime. It even outperforms handwritten code using the ABI interfaces directly. That's because the abstractions use modern C++ idioms that the Visual C++ compiler is designed to optimize. This includes magic statics, empty base classes, **strlen** elision, as well as many newer optimizations in the latest version of Visual C++ targeted specifically at improving the performance of C++/WinRT.

There are ways to gradually introduce C++/WinRT into your projects. You could use Windows Runtime components, or you could interoperate with C++/CX. For more info, see Interop between C++/WinRT and C++/CX.

For info about porting to C++/WinRT, see these resources.

- Move to C++/WinRT from C++/CX
- Move to C++/WinRT from WRL
- Move to C++/WinRT from C#

Also see Where can I find C++/WinRT sample apps?.

Topics about C++/WinRT

Topic	Description
Introduction to C++/WinRT	An introduction to C++/WinRT—a standard C++ language projection for Windows Runtime APIs.
Get started with C++/WinRT	To get you up to speed with using C++/WinRT, this topic walks through a simple code example.
What's new in C++/WinRT	News and changes to C++/WinRT.
Frequently-asked questions	Answers to questions that you're likely to have about authoring and consuming Windows Runtime APIs with C++/WinRT.
Troubleshooting	The table of troubleshooting symptoms and remedies in this topic may be helpful to you whether you're cutting new code or porting an existing app.
Photo Editor C++/WinRT sample application	Photo Editor is a UWP sample application that showcases development with the C++/WinRT language projection. The sample application allows you to retrieve photos from the **Pictures** library, and then edit the selected image with assorted photo effects.
String handling	With C++/WinRT, you can call Windows Runtime APIs using standard C++ wide string types, or you can use the

Topic	Description
	winrt::hstring type.
Standard C++ data types and C++/WinRT	With C++/WinRT, you can call Windows Runtime APIs using Standard C++ data types.
Boxing and unboxing values to IInspectable	A scalar or array value needs to be wrapped inside a reference class object before being passed to a function that expects **IInspectable**. That wrapping process is known as *boxing* the value.
Consume APIs with C++/WinRT	This topic shows how to consume C++/WinRT APIs, whether they're implemented by Windows, a third-party component vendor, or by yourself.
Author APIs with C++/WinRT	This topic shows how to author C++/WinRT APIs by using the **winrt::implements** base struct, either directly or indirectly.
Error handling with C++/WinRT	This topic discusses strategies for handling errors when programming with C++/WinRT.
Handle events by using delegates	This topic shows how to register and revoke event-handling delegates using C++/WinRT.
Author events	This topic demonstrates how to author a Windows Runtime component containing a runtime class that raises events. It also demonstrates an app that consumes the component and handles the events.
Collections with C++/WinRT	C++/WinRT provides functions and base classes that save you a lot of time and effort when you want to implement and/or pass collections.
Concurrency and asynchronous operations	This topic shows the ways in which you can both create and consume Windows Runtime asynchronous objects with C++/WinRT.
Advanced concurrency and asynchrony	Advanced scenarios with concurrency and asynchrony in C++/WinRT.
A completion source sample	Shows how you can author and consume your own completion source class.
XAML controls; bind to a C++/WinRT property	A property that can be effectively bound to a XAML control is known as an *observable* property. This topic shows how to implement and consume an observable property, and how to bind a XAML control to it.
XAML items controls; bind to a C++/WinRT collection	A collection that can be effectively bound to a XAML items control is known as an *observable* collection. This topic shows

Topic	Description
	how to implement and consume an observable collection, and how to bind a XAML items control to it.
XAML custom (templated) controls with C++/WinRT	This topic walks you through the steps of creating a simple custom control using C++/WinRT. You can build on the info here to create your own feature-rich and customizable UI controls.
Passing parameters to projected APIs	C++/WinRT simplifies passing parameters to projected APIs by providing automatic conversions for common cases.
Consume COM components with C++/WinRT	This topic uses a full Direct2D code example to show how to use C++/WinRT to consume COM classes and interfaces.
Author COM components with C++/WinRT	C++/WinRT can help you to author classic COM components, just as it helps you to author Windows Runtime classes.
Move to C++/WinRT from C++/CX	This topic describes the technical details involved in porting the source code in a C++/CX project to its equivalent in C++/WinRT.
Interop between C++/WinRT and C++/CX	This topic shows two helper functions that can be used to convert between C++/CX and C++/WinRT objects.
Asynchrony, and interop between C++/WinRT and C++/CX	This is an advanced topic related to gradually porting from C++/CX to C++/WinRT. It shows how Parallel Patterns Library (PPL) tasks and coroutines can exist side by side in the same project.
Move to C++/WinRT from WRL	This topic shows how to port Windows Runtime C++ Template Library (WRL) code to its equivalent in C++/WinRT.
Porting the Clipboard sample to C++/WinRT from C#—a case study	This topic presents a case study of porting one of the Universal Windows Platform (UWP) app samples from C# to C++/WinRT. You can gain porting practice and experience by following along with the walkthrough and porting the sample for yourself as you go.
Move to C++/WinRT from C#	This topic comprehensively catalogs the technical details involved in porting the source code in a C# project to its equivalent in C++/WinRT.
Interop between C++/WinRT and the ABI	This topic shows how to convert between application binary interface (ABI) and C++/WinRT objects.
Strong and weak references in C++/WinRT	The Windows Runtime is a reference-counted system; and in such a system it's important for you to know about the significance of, and distinction between, strong and weak references.

Topic	Description
Agile objects	An agile object is one that can be accessed from any thread. Your C++/WinRT types are agile by default, but you can opt out.
Diagnosing direct allocations	This topic goes in-depth on a C++/WinRT 2.0 feature that helps you diagnose the mistake of creating an object of implementation type on the stack, rather than using the **winrt::make** family of helpers, as you should.
Extension points for your implementation types	These extension points in C++/WinRT 2.0 allow you to defer destruction of your implementation types, to safely query during destruction, and to hook the entry into and exit from your projected methods.
A basic C++/WinRT Windows UI Library 2 example (UWP)	This topic walks you through the process of adding basic support for the Windows UI Library (WinUI) to your C++/WinRT UWP project. Specifically, this topic deals with WinUI 2, which is for UWP apps.
Windows Runtime components with C++/WinRT	This topic shows how to use C++/WinRT to create and consume a Windows Runtime component—a component that's callable from a Universal Windows app built using any Windows Runtime language.
Authoring a C# Windows Runtime component for use from a C++/WinRT app	This topic walks you through the process of adding a simple C# component to your C++/WinRT project.
Visual Studio native debug visualization (natvis) for C++/WinRT	The C++/WinRT Visual Studio Extension (VSIX) gives you Visual Studio native debug visualization (natvis) of C++/WinRT projected types. This provides you an experience similar to C# debugging.
Configuration macros	This topic describes the C++/WinRT configuration macros.
C++/WinRT naming conventions	This topic explains naming conventions that C++/WinRT has established.

Topics about the C++ language

Topic	Description
Value categories, and references to them	This topic describes the various categories of values that exist in C++. You will doubtless have heard of lvalues and rvalues, but there are other kinds, too.

Important APIs

- winrt namespace

Related topics

- C++/CX
- Windows Runtime C++ Template Library (WRL)

Introduction to C++/WinRT

Article • 11/18/2022

https://www.youtube-nocookie.com/embed/X41j_gzSwOY

https://www.youtube-nocookie.com/embed/nOFNc2uTmGs

C++/WinRT is an entirely standard modern C++17 language projection for Windows Runtime (WinRT) APIs, implemented as a header-file-based library, and designed to provide you with first-class access to the modern Windows API. With C++/WinRT, you can author and consume Windows Runtime APIs using any standards-compliant C++17 compiler. The Windows SDK includes C++/WinRT; it was introduced in version 10.0.17134.0 (Windows 10, version 1803).

C++/WinRT is Microsoft's recommended replacement for the C++/CX language projection, and the Windows Runtime C++ Template Library (WRL). The full list of topics about C++/WinRT includes info about both interoperating with, and porting from, C++/CX and WRL.

> ⓘ **Important**
>
> Some of the most important pieces of C++/WinRT to be aware of are described in the sections **SDK support for C++/WinRT** and **Visual Studio support for C++/WinRT, XAML, the VSIX extension, and the NuGet package**.

Also see Where can I find C++/WinRT sample apps?.

Language projections

The Windows Runtime is based on Component Object Model (COM) APIs, and it's designed to be accessed through *language projections*. A projection hides the COM details, and provides a more natural programming experience for a given language.

The C++/WinRT language projection in the Windows Runtime API reference content

When you're browsing Windows Runtime APIs, click the **Language** combo box in the upper right, and select **C++/WinRT** to view API syntax blocks as they appear in the

C++/WinRT language projection.

Visual Studio support for C++/WinRT, XAML, the VSIX extension, and the NuGet package

For Visual Studio support, you'll need Visual Studio 2022, or Visual Studio 2019, or Visual Studio 2017 (at least version 15.6; we recommend at least 15.7). From within the Visual Studio Installer, install the **Universal Windows Platform development** workload. In **Installation Details** > **Universal Windows Platform development**, check the **C++ (v14x) Universal Windows Platform tools** option(s) if you haven't already done so. And, in Windows **Settings** > **Privacy & security** (**Windows 10**: **Update & Security**) > **For developers**, enable the **Developer mode** option (**Windows 10**: not the **Sideload apps** option).

While we recommend that you develop with the latest versions of Visual Studio and the Windows SDK, if you're using a version of C++/WinRT that shipped with the Windows SDK prior to 10.0.17763.0 (Windows 10, version 1809), then, to use the Windows namespaces headers mentioned above, you'll need a minimum Windows SDK target version in your project of 10.0.17134.0 (Windows 10, version 1803).

Visual Studio 2022 ships with C++/WinRT project and item templates built in, so that you can get started with C++/WinRT development right away. It also comes with Visual Studio native debug visualization (natvis) of C++/WinRT projected types; providing an experience similar to C# debugging. Natvis is automatic for debug builds. For more info, see Visual Studio native debug visualization for C++/WinRT.

For older versions of Visual Studio, you'll want to download and install the latest version of the C++/WinRT Visual Studio Extension (VSIX) from the Visual Studio Marketplace.

- The VSIX extension gives you C++/WinRT project and item templates in Visual Studio.
- In addition, it gives you Visual Studio native debug visualization (natvis) of C++/WinRT projected types.

The Visual Studio project templates for C++/WinRT are described in the sections below. When you create a new C++/WinRT project with the latest version of the VSIX extension installed, the new C++/WinRT project automatically installs the Microsoft.Windows.CppWinRT NuGet package. The **Microsoft.Windows.CppWinRT** NuGet package provides C++/WinRT build support (MSBuild properties and targets), making your project portable between a development machine and a build agent (on which only the NuGet package, and not the VSIX extension, is installed).

Alternatively, you can convert an existing project by manually installing the **Microsoft.Windows.CppWinRT** NuGet package. After installing (or updating to) the latest version of the VSIX extension, open the existing project in Visual Studio, click **Project** > **Manage NuGet Packages...** > **Browse**, type or paste **Microsoft.Windows.CppWinRT** in the search box, select the item in search results, and then click **Install** to install the package for that project. Once you've added the package, you'll get C++/WinRT MSBuild support for the project, including invoking the `cppwinrt.exe` tool.

> **ⓘ Important**
>
> If you have projects that were created with (or upgraded to work with) a version of the VSIX extension earlier than 1.0.190128.4, then see **Earlier versions of the VSIX extension**. That section contains important info about the configuration of your projects, which you'll need to know to upgrade them to use the latest version of the VSIX extension.

- Because C++/WinRT uses features from the C++17 standard, the NuGet package sets project property **C/C++** > **Language** > **C++ Language Standard** > **ISO C++17 Standard (/std:c++17)** in Visual Studio.
- It also adds the /bigobj compiler option.
- It adds the /await compiler option in order to enable `co_await`.
- It instructs the XAML compiler to emit C++/WinRT codegen.
- You might also want to set **Conformance mode: Yes (/permissive-)**, which further constrains your code to be standards-compliant.
- Another project property to be aware of is **C/C++** > **General** > **Treat Warnings As Errors**. Set this to **Yes(/WX)** or **No (/WX-)** to taste. Sometimes, source files generated by the `cppwinrt.exe` tool generate warnings until you add your implementation to them.

With your system set up as described above, you'll be able to create and build, or open, a C++/WinRT project in Visual Studio, and deploy it.

As of version 2.0, the **Microsoft.Windows.CppWinRT** NuGet package includes the `cppwinrt.exe` tool. You can point the `cppwinrt.exe` tool at a Windows Runtime metadata (`.winmd`) file to generate a header-file-based standard C++ library that *projects* the APIs described in the metadata for consumption from C++/WinRT code. Windows Runtime metadata (`.winmd`) files provide a canonical way of describing a Windows Runtime API surface. By pointing `cppwinrt.exe` at metadata, you can generate a library for use with any runtime class implemented in a second- or third-party

Windows Runtime component, or implemented in your own application. For more info, see Consume APIs with C++/WinRT.

With C++/WinRT, you can also implement your own runtime classes using standard C++, without resorting to COM-style programming. For a runtime class, you just describe your types in an IDL file, and `midl.exe` and `cppwinrt.exe` generate your implementation boilerplate source code files for you. You can alternatively just implement interfaces by deriving from a C++/WinRT base class. For more info, see Author APIs with C++/WinRT.

For a list of customization options for the `cppwinrt.exe` tool, set via project properties, see the Microsoft.Windows.CppWinRT NuGet package readme.

You can identify a project that uses the C++/WinRT MSBuild support by the presence of the **Microsoft.Windows.CppWinRT** NuGet package installed within the project.

Here are the Visual Studio project templates provided by the VSIX extension.

Blank App (C++/WinRT)

A project template for a Universal Windows Platform (UWP) app that has a XAML user-interface.

Visual Studio provides XAML compiler support to generate implementation and header stubs from the Interface Definition Language (IDL) (`.idl`) file that sits behind each XAML markup file. In an IDL file, define any local runtime classes that you want to reference in your app's XAML pages, and then build the project once to generate implementation templates in `Generated Files`, and stub type definitions in `Generated Files\sources`. Then use those stub type definitions for reference to implement your local runtime classes. See Factoring runtime classes into Midl files (.idl).

The XAML design surface support in Visual Studio for C++/WinRT is close to parity with C#. In Visual Studio, you can use the **Events** tab of the **Properties** window to add event handlers within a C++/WinRT project. You can also add event handlers to your code manually—see Handle events by using delegates in C++/WinRT for more info.

Core App (C++/WinRT)

A project template for a Universal Windows Platform (UWP) app that doesn't use XAML.

Instead, it uses the C++/WinRT Windows namespace header for the Windows.ApplicationModel.Core namespace. After building and running, click on an empty space to add a colored square; then click on a colored square to drag it.

Windows Console Application (C++/WinRT)

A project template for a C++/WinRT client application for Windows Desktop, with a console user-interface.

Windows Desktop Application (C++/WinRT)

A project template for a C++/WinRT client application for Windows Desktop, which displays a Windows Runtime Windows.Foundation.Uri inside a Win32 **MessageBox**.

Windows Runtime Component (C++/WinRT)

A project template for a component; typically for consumption from a Universal Windows Platform (UWP).

This template demonstrates the `midl.exe` > `cppwinrt.exe` toolchain, where Windows Runtime metadata (`.winmd`) is generated from IDL, and then implementation and header stubs are generated from the Windows Runtime metadata.

In an IDL file, define the runtime classes in your component, their default interface, and any other interfaces they implement. Build the project once to generate `module.g.cpp`, `module.h.cpp`, implementation templates in `Generated Files`, and stub type definitions in `Generated Files\sources`. Then use those the stub type definitions for reference to implement the runtime classes in your component. See Factoring runtime classes into Midl files (.idl).

Bundle the built Windows Runtime component binary and its `.winmd` with the UWP app consuming them.

Earlier versions of the VSIX extension

We recommend that you install (or update to) the latest version of the VSIX extension. It is configured to update itself by default. If you do that, and you have projects that were created with a version of the VSIX extension earlier than 1.0.190128.4, then this section contains important info about upgrading those projects to work with the new version. If you don't update, then you'll still find the info in this section useful.

In terms of supported Windows SDK and Visual Studio versions, and Visual Studio configuration, the info in the Visual Studio support for C++/WinRT, XAML, the VSIX extension, and the NuGet package section above applies to earlier versions of the VSIX

extension. The info below describes important differences regarding the behavior and configuration of projects created with (or upgraded to work with) earlier versions.

Created earlier than 1.0.181002.2

If your project was created with a version of the VSIX extension earlier than 1.0.181002.2, then C++/WinRT build support was built into that version of the VSIX extension. Your project has the `<CppWinRTEnabled>true</CppWinRTEnabled>` property set in the `.vcxproj` file.

XML

```
<Project ...>
    <PropertyGroup Label="Globals">
        <CppWinRTEnabled>true</CppWinRTEnabled>
...
```

You can upgrade your project by manually installing the **Microsoft.Windows.CppWinRT** NuGet package. After installing (or upgrading to) the latest version of the VSIX extension, open your project in Visual Studio, click **Project** > **Manage NuGet Packages...** > **Browse**, type or paste **Microsoft.Windows.CppWinRT** in the search box, select the item in search results, and then click **Install** to install the package for your project.

Created with (or upgraded to) between 1.0.181002.2 and 1.0.190128.3

If your project was created with a version of the VSIX extension between 1.0.181002.2 and 1.0.190128.3, inclusive, then the **Microsoft.Windows.CppWinRT** NuGet package was installed in the project automatically by the project template. You might also have upgraded an older project to use a version of the VSIX extension in this range. If you did, then—since build support was also still present in versions of the VSIX extension in this range—your upgraded project may or may not have the **Microsoft.Windows.CppWinRT** NuGet package installed.

To upgrade your project, follow the instructions in the previous section and ensure that your project does have the **Microsoft.Windows.CppWinRT** NuGet package installed.

Invalid upgrade configurations

With the latest version of the VSIX extension, it's not valid for a project to have the `<CppWinRTEnabled>true</CppWinRTEnabled>` property if it doesn't also have the

Microsoft.Windows.CppWinRT NuGet package installed. A project with this configuration produces the build error message, "The C++/WinRT VSIX no longer provides project build support. Please add a project reference to the Microsoft.Windows.CppWinRT Nuget package."

As mentioned above, a C++/WinRT project now needs to have the NuGet package installed in it.

Since the `<CppWinRTEnabled>` element is now obsolete, you can optionally edit your `.vcxproj`, and delete the element. It's not strictly necessary, but it's an option.

Also, if your `.vcxproj` contains `<RequiredBundles>$(RequiredBundles);Microsoft.Windows.CppWinRT</RequiredBundles>`, then you can remove it so that you can build without requiring the C++/WinRT VSIX extension to be installed.

SDK support for C++/WinRT

Although it is now present only for compatibility reasons, as of version 10.0.17134.0 (Windows 10, version 1803), the Windows SDK contains a header-file-based standard C++ library for consuming first-party Windows APIs (Windows Runtime APIs in Windows namespaces). Those headers are inside the folder `%WindowsSdkDir%Include<WindowsTargetPlatformVersion>\cppwinrt\winrt`. As of the Windows SDK version 10.0.17763.0 (Windows 10, version 1809), these headers are generated for you inside your project's *$(GeneratedFilesDir)* folder.

Again for compatibility, the Windows SDK also comes with the `cppwinrt.exe` tool. However, we recommend that you instead install and use the most recent version of `cppwinrt.exe`, which is included with the **Microsoft.Windows.CppWinRT** NuGet package. That package, and `cppwinrt.exe`, are described in the sections above.

Custom types in the C++/WinRT projection

In your C++/WinRT programming, you can use standard C++ language features and Standard C++ data types and C++/WinRT—including some C++ Standard Library data types. But you'll also become aware of some custom data types in the projection, and you can choose to use them. For example, we use **winrt::hstring** in the quick-start code example in Get started with C++/WinRT.

winrt::com_array is another type that you're likely to use at some point. But you're less likely to directly use a type such as **winrt::array_view**. Or you may choose not to use it

so that you won't have any code to change if and when an equivalent type appears in the C++ Standard Library.

> ⚠ **Warning**
>
> There are also types that you might see if you closely study the C++/WinRT Windows namespace headers. An example is **winrt::param::hstring**, but there are collection examples too. These exist solely to optimize the binding of input parameters, and they yield big performance improvements and make most calling patterns "just work" for related standard C++ types and containers. These types are only ever used by the projection in cases where they add most value. They're highly optimized and they're not for general use; don't be tempted to use them yourself. Nor should you use anything from the `winrt::impl` namespace, since those are implementation types, and therefore subject to change. You should continue to use standard types, or types from the **winrt namespace**.
>
> Also see **Passing parameters into the ABI boundary**.

Important APIs

- winrt::hstring struct
- winrt namespace

Related topics

- C++/CX
- C++/WinRT Visual Studio Extension (VSIX)
- Get started with C++/WinRT
- Standard C++ data types and C++/WinRT
- String handling in C++/WinRT
- Windows Runtime APIs

Get started with C++/WinRT

Article • 02/13/2023

> **Important**
>
> For info about setting up Visual Studio for C++/WinRT development—including installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support)—see **Visual Studio support for C++/WinRT**.

To get you up to speed with using C++/WinRT, this topic walks through a simple code example based on a new **Windows Console Application (C++/WinRT)** project. This topic also shows how to add C++/WinRT support to a Windows Desktop application project.

> **Note**
>
> While we recommend that you develop with the latest versions of Visual Studio and the Windows SDK, if you're using Visual Studio 2017 (version 15.8.0 or later), and targeting the Windows SDK version 10.0.17134.0 (Windows 10, version 1803), then a newly created C++/WinRT project may fail to compile with the error "*error C3861: 'from_abi': identifier not found*", and with other errors originating in *base.h*. The solution is to either target a later (more conformant) version of the Windows SDK, or set project property **C/C++** > **Language** > **Conformance mode: No** (also, if **/permissive-** appears in project property **C/C++** > **Language** > **Command Line** under **Additional Options**, then delete it).

A C++/WinRT quick-start

Create a new **Windows Console Application (C++/WinRT)** project.

Edit `pch.h` and `main.cpp` to look like this.

C++/WinRT

```
// pch.h
#pragma once
#include <winrt/Windows.Foundation.Collections.h>
```

```
#include <winrt/Windows.Web.Syndication.h>
#include <iostream>
```

C++/WinRT

```
// main.cpp
#include "pch.h"

using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;

int main()
{
    winrt::init_apartment();

    Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
    SyndicationClient syndicationClient;
    syndicationClient.SetRequestHeader(L"User-Agent", L"Mozilla/5.0
(compatible; MSIE 10.0; Windows NT 6.2; WOW64; Trident/6.0)");
    SyndicationFeed syndicationFeed =
syndicationClient.RetrieveFeedAsync(rssFeedUri).get();
    for (const SyndicationItem syndicationItem : syndicationFeed.Items())
    {
        winrt::hstring titleAsHstring = syndicationItem.Title().Text();

        // A workaround to remove the trademark symbol from the title
string, because it causes issues in this case.
        std::wstring titleAsStdWstring{ titleAsHstring.c_str() };
        titleAsStdWstring.erase(remove(titleAsStdWstring.begin(),
titleAsStdWstring.end(), L'™'), titleAsStdWstring.end());
        titleAsHstring = titleAsStdWstring;

        std::wcout << titleAsHstring.c_str() << std::endl;
    }
}
```

Let's take the short code example above piece by piece, and explain what's going on in each part.

C++/WinRT

```
#include <winrt/Windows.Foundation.Collections.h>
#include <winrt/Windows.Web.Syndication.h>
```

With the default project settings, the included headers come from the Windows SDK, inside the folder `%WindowsSdkDir%Include<WindowsTargetPlatformVersion>\cppwinrt\winrt`. Visual Studio includes that path in its *IncludePath* macro. But there's no strict dependency on the

Windows SDK, because your project (via the `cppwinrt.exe` tool) generates those same headers into your project's *$(GeneratedFilesDir)* folder. They'll be loaded from that folder if they can't be found elsewhere, or if you change your project settings.

The headers contain Windows APIs projected into C++/WinRT. In other words, for each Windows type, C++/WinRT defines a C++-friendly equivalent (called the *projected type*). A projected type has the same fully-qualified name as the Windows type, but it's placed in the C++ **winrt** namespace. Putting these includes in your precompiled header reduces incremental build times.

> **ⓘ Important**
>
> Whenever you want to use a type from a Windows namespaces, you must `#include` the corresponding C++/WinRT Windows namespace header file, as shown above. The *corresponding* header is the one with the same name as the type's namespace. For example, to use the C++/WinRT projection for the Windows::Foundation::Collections::PropertySet runtime class, include the `winrt/Windows.Foundation.Collections.h` header.
>
> It is common for a C++/WinRT projection header to automatically include related namespace header files. For example, `winrt/Windows.Foundation.Collections.h` includes `winrt/Windows.Foundation.h`. But you shouldn't rely on this behavior, since it's an implementation detail that changes over time. You must explicitly include any headers that you need.

C++/WinRT

```
using namespace winrt;
using namespace Windows::Foundation;
using namespace Windows::Web::Syndication;
```

The `using namespace` directives are optional, but convenient. The pattern shown above for such directives (allowing unqualified name lookup for anything in the **winrt** namespace) is suitable for when you're beginning a new project and C++/WinRT is the only language projection you're using inside of that project. If, on the other hand, you're mixing C++/WinRT code with C++/CX and/or SDK application binary interface (ABI) code (you're either porting from, or interoperating with, one or both of those models), then see the topics Interop between C++/WinRT and C++/CX, Move to C++/WinRT from C++/CX, and Interop between C++/WinRT and the ABI.

C++/WinRT

```
winrt::init_apartment();
```

The call to **winrt::init_apartment** initializes the thread in the Windows Runtime; by default, in a multithreaded apartment. The call also initializes COM.

C++/WinRT

```
Uri rssFeedUri{ L"https://blogs.windows.com/feed" };
SyndicationClient syndicationClient;
```

Stack-allocate two objects: they represent the uri of the Windows blog, and a syndication client. We construct the uri with a simple wide string literal (see String handling in C++/WinRT for more ways you can work with strings).

C++/WinRT

```
SyndicationFeed syndicationFeed =
syndicationClient.RetrieveFeedAsync(rssFeedUri).get();
```

SyndicationClient::RetrieveFeedAsync is an example of an asynchronous Windows Runtime function. The code example receives an asynchronous operation object from **RetrieveFeedAsync**, and it calls **get** on that object to block the calling thread and wait for the result (which is a syndication feed, in this case). For more about concurrency, and for non-blocking techniques, see Concurrency and asynchronous operations with C++/WinRT.

C++/WinRT

```
for (const SyndicationItem syndicationItem : syndicationFeed.Items()) { ...
}
```

SyndicationFeed.Items is a range, defined by the iterators returned from **begin** and **end** functions (or their constant, reverse, and constant-reverse variants). Because of this, you can enumerate **Items** with either a range-based `for` statement, or with the **std::for_each** template function. Whenever you iterate over a Windows Runtime collection like this, you'll need to `#include <winrt/Windows.Foundation.Collections.h>`.

C++/WinRT

```
winrt::hstring titleAsHstring = syndicationItem.Title().Text();

// Omitted: there's a little bit of extra work here to remove the trademark
symbol from the title text.
```

```
std::wcout << titleAsHstring.c_str() << std::endl;
```

Gets the feed's title text, as a **winrt::hstring** object (more details in String handling in C++/WinRT). The **hstring** is then output, via the **c_str** function, which reflects the pattern used with C++ Standard Library strings.

As you can see, C++/WinRT encourages modern, and class-like, C++ expressions such as `syndicationItem.Title().Text()`. This is a different, and cleaner, programming style from traditional COM programming. You don't need to directly initialize COM, nor work with COM pointers.

Nor do you need to handle HRESULT return codes. C++/WinRT converts error HRESULTs to exceptions such as **winrt::hresult-error** for a natural and modern programming style. For more info about error-handling, and code examples, see Error handling with C++/WinRT.

Modify a Windows Desktop application project to add C++/WinRT support

Some desktop projects (for example, the WinUI 3 templates in Visual Studio) have C++/WinRT support built in.

But this section shows you how you can add C++/WinRT support to any Windows Desktop application project that you might have. If you don't have an existing Windows Desktop application project, then you can follow along with these steps by first creating one. For example, open Visual Studio and create a **Visual C++** > **Windows Desktop** > **Windows Desktop Application** project.

You can optionally install the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package. For details, see Visual Studio support for C++/WinRT.

Set project properties

Go to project property **General** > **Windows SDK Version**, and select **All Configurations** and **All Platforms**. Ensure that **Windows SDK Version** is set to 10.0.17134.0 (Windows 10, version 1803) or greater.

Confirm that you're not affected by Why won't my new project compile?.

Because C++/WinRT uses features from the C++17 standard, set project property **C/C++** > **Language** > **C++ Language Standard** to *ISO C++17 Standard (/std:c++17)*.

The precompiled header

The default project template creates a precompiled header for you, named either `framework.h`, or `stdafx.h`. Rename that to `pch.h`. If you have a `stdafx.cpp` file, then rename that to `pch.cpp`. Set project property **C/C++** > **Precompiled Headers** > **Precompiled Header** to *Create (/Yc)*, and **Precompiled Header File** to *pch.h*.

Find and replace all `#include "framework.h"` (or `#include "stdafx.h"`) with `#include "pch.h"`.

In `pch.h`, include `winrt/base.h`.

C++/WinRT

```
// pch.h
...
#include <winrt/base.h>
```

Linking

The C++/WinRT language projection depends on certain Windows Runtime free (non-member) functions, and entry points, that require linking to the WindowsApp.lib umbrella library. This section describes three ways of satisfying the linker.

The first option is to add to your Visual Studio project all of the C++/WinRT MSBuild properties and targets. To do this, install the Microsoft.Windows.CppWinRT NuGet package into your project. Open the project in Visual Studio, click **Project** > **Manage NuGet Packages...** > **Browse**, type or paste **Microsoft.Windows.CppWinRT** in the search box, select the item in search results, and then click **Install** to install the package for that project.

You can also use project link settings to explicitly link `WindowsApp.lib`. Or, you can do it in source code (in `pch.h`, for example) like this.

C++/WinRT

```
#pragma comment(lib, "windowsapp")
```

You can now compile and link, and add C++/WinRT code to your project (for example, code similar to that shown in the A C++/WinRT quick-start section, above).

The three main scenarios for C++/WinRT

As you use and become familiar with C++/WinRT, and work through the rest of the documentation here, you'll likely notice that there are three main scenarios, as described in the following sections.

Consuming Windows APIs and types

In other words, *using*, or *calling* APIs. For example, making API calls to communicate using Bluetooth; to stream and present video; to integrate with the Windows shell; and so on. C++/WinRT fully and uncompromisingly supports this category of scenario. For more info, see Consume APIs with C++/WinRT.

Authoring Windows APIs and types

In other words, *producing* APIs and types. For example, producing the kinds of APIs described in the section above; or the graphics APIs; the storage and file system APIs; the networking APIs, and so on. For more info, see Author APIs with C++/WinRT.

Authoring APIs with C++/WinRT is a little more involved than consuming them, because you must use IDL to define the shape of the API before you can implement it. There's a walkthrough of doing that in XAML controls; bind to a C++/WinRT property.

XAML applications

This scenario is about building applications and controls on the XAML UI framework. Working in a XAML application amounts to a combination of consuming and authoring. But since XAML is the dominant UI framework on Windows today, and its influence over the Windows Runtime is proportionate to that, it deserves its own category of scenario.

Be aware that XAML works best with programming languages that offer reflection. In C++/WinRT, you sometimes have to do a little extra work in order to interoperate with the XAML framework. All of those cases are covered in the documentation. Good places to start are XAML controls; bind to a C++/WinRT property and XAML custom (templated) controls with C++/WinRT.

Sample apps written in C++/WinRT

See Where can I find C++/WinRT sample apps?.

Important APIs

- SyndicationClient::RetrieveFeedAsync method
- SyndicationFeed.Items property
- winrt::hstring struct
- winrt::hresult-error struct

Related topics

- C++/CX
- Error handling with C++/WinRT
- Interop between C++/WinRT and C++/CX
- Interop between C++/WinRT and the ABI
- Move to C++/WinRT from C++/CX
- String handling in C++/WinRT

Feedback

What's new in C++/WinRT

Article • 10/27/2022

As subsequent versions of C++/WinRT are released, this topic describes what's new, and what's changed.

Rollup of recent improvements/additions as of March 2020

Up to 23% shorter build times

The C++/WinRT and C++ compiler teams have collaborated to do everything possible to shorten build times. We've pored over compiler analytics to figure out how the internals of C++/WinRT can be restructured to help the C++ compiler eliminate compile-time overhead, as well as how the C++ compiler itself can be improved to handle the C++/WinRT library. C++/WinRT has been optimized for the compiler; and the compiler has been optimized for C++/WinRT.

Let's take for example the worst case scenario of building a pre-compiled header (PCH) that contains every single C++/WinRT projection namespace header.

Expand table

Version	PCH size (bytes)	Time (s)
C++/WinRT from July, with Visual C++ 16.3	3,004,104,632	31
version 2.0.200316.3 of C++/WinRT, with Visual C++ 16.5	2,393,515,336	24

A 20% reduction in size, and a 23% reduction in build time.

Improved MSBuild support

We've invested a lot of work into improving MSBuild support for a large selection of different scenarios.

Even faster factory caching

We've improved inlining of the factory cache in order to better inline hot paths, which leads to faster execution.

That improvement doesn't affect code size—as described below in Optimized EH code-gen, if your application uses C++ exception handling heavily, then you can shrink your binary by using the `/d2FH4` option, which is on by default in new projects created with Visual Studio 2019 16.3, and later.

More efficient boxing

When used in a XAML application, **winrt::box_value** is now more efficient (see Boxing and unboxing). Applications that do a lot of boxing will also notice a reduction in code size.

Support for implementing COM interfaces that implement IInspectable

If you need to implement a (non-Windows-Runtime) COM interface that just happens to implement **IInspectable**, then you can now do so with C++/WinRT. See COM interfaces that implement IInspectable.

Module-locking improvements

Control over module-locking now allows both custom hosting scenarios and the elimination of module-level locking altogether. See Module-locking improvements.

Support for non-Windows-Runtime error information

Some APIs (even some Windows Runtime APIs) report errors without using Windows Runtime error origination APIs. In cases like that, C++/WinRT now falls back to using COM error info. See C++/WinRT support for non-WinRT error information.

Enable C++ module support

C++ module support is back, but only in an experimental form. The feature isn't complete in the C++ compiler, as yet.

More efficient coroutine resumption

C++/WinRT coroutines already perform well, but we continue to look for ways to improve that. See Improve scalability of coroutine resumption.

New when_all and when_any async helpers

The **when_all** helper function creates an **IAsyncAction** object that completes when all of the supplied awaitables have completed. The **when_any** helper creates an **IAsyncAction** that completes when any of the supplied awaitables have completed.

See Add when_any async helper and Add when_all async helper.

Other optimizations and additions

In addition, many bug fixes and minor optimizations and additions have been introduced, including various improvements to simplify debugging and to optimize internals and default implementations. Follow this link for an exhaustive list: https://github.com/microsoft/xlang/pulls?q=is%3Apr+is%3Aclosed.

News, and changes, in C++/WinRT 2.0

For more info about the C++/WinRT Visual Studio Extension (VSIX), the Microsoft.Windows.CppWinRT NuGet package, and the `cppwinrt.exe` tool—including how to acquire and install them—see Visual Studio support for C++/WinRT, XAML, the VSIX extension, and the NuGet package.

Changes to the C++/WinRT Visual Studio Extension (VSIX) for version 2.0

- The debug visualizer now supports Visual Studio 2019; as well as continuing to support Visual Studio 2017.
- Numerous bug fixes have been made.

Changes to the Microsoft.Windows.CppWinRT NuGet package for version 2.0

- The `cppwinrt.exe` tool is now included in the Microsoft.Windows.CppWinRT NuGet package, and the tool generates platfom projection headers for each project on demand. Consequently, the `cppwinrt.exe` tool no longer depends on the Windows SDK (although, the tool still ships with the SDK for compatibility reasons).
- `cppwinrt.exe` now generates projection headers under each platform/configuration-specific intermediate folder ($IntDir) to enable parallel builds.

- The C++/WinRT build support (props/targets) is now fully documented, in case you want to manually customize your project files. See the Microsoft.Windows.CppWinRT NuGet package readme.
- Numerous bug fixes have been made.

Changes to C++/WinRT for version 2.0

Open source

The `cppwinrt.exe` tool takes a Windows Runtime metadata (`.winmd`) file, and generates from it a header-file-based standard C++ library that *projects* the APIs described in the metadata. That way, you can consume those APIs from your C++/WinRT code.

This tool is now an entirely open source project, available on GitHub. Visit Microsoft/cppwinrt.

xlang libraries

A completely portable header-only library (for parsing the ECMA-335 metadata format used by the Windows Runtime) forms the basis of all the Windows Runtime and xlang tooling going forward. Notably, we also rewrote the `cppwinrt.exe` tool from the ground up using the xlang libraries. This provides far more accurate metadata queries, solving a few long-standing issues with the C++/WinRT language projection.

Fewer dependencies

Due to the xlang metadata reader, the `cppwinrt.exe` tool itself has fewer dependencies. This makes it far more flexible, as well as being usable in more scenarios—especially in constrained build environments. Notably, it no longer relies on `RoMetadata.dll`. These are the dependencies for `cppwinrt.exe` 2.0.

- ADVAPI32.dll
- KERNEL32.dll
- SHLWAPI.dll
- XmlLite.dll

All of those DLLs are available not only on Windows 10, but all the way down to Windows 7, and even Windows Vista. If you want it to, your old build server running Windows 7 can now run `cppwinrt.exe` to generate C++ headers for your project. With a bit of work, you can even run C++/WinRT on Windows 7, if that interests you.

Contrast the list above with these dependencies, which `cppwinrt.exe` 1.0 has.

- ADVAPI32.dll
- SHELL32.dll
- api-ms-win-core-file-l1-1-0.dll
- XmlLite.dll
- api-ms-win-core-libraryloader-l1-2-0.dll
- api-ms-win-core-processenvironment-l1-1-0.dll
- RoMetadata.dll
- SHLWAPI.dll
- KERNEL32.dll
- api-ms-win-core-rtlsupport-l1-1-0.dll
- api-ms-win-core-heap-l1-1-0.dll
- api-ms-win-core-timezone-l1-1-0.dll
- api-ms-win-core-console-l1-1-0.dll
- api-ms-win-core-localization-l1-2-0.dll
- OLEAUT32.dll
- api-ms-win-core-winrt-error-l1-1-0.dll
- api-ms-win-core-winrt-error-l1-1-1.dll
- api-ms-win-core-winrt-l1-1-0.dll
- api-ms-win-core-winrt-string-l1-1-0.dll
- api-ms-win-core-synch-l1-1-0.dll
- api-ms-win-core-threadpool-l1-2-0.dll
- api-ms-win-core-com-l1-1-0.dll
- api-ms-win-core-com-l1-1-1.dll
- api-ms-win-core-synch-l1-2-0.dll

The Windows Runtime `noexcept` attribute

The Windows Runtime has a new `[noexcept]` attribute, which you may use to decorate your methods and properties in MIDL 3.0. The presence of the attribute indicates to supporting tools that your implementation doesn't throw an exception (nor return a failing HRESULT). This allows language projections to optimize code-generation by avoiding the exception-handling overhead that's required to support application binary interface (ABI) calls that can potentially fail.

C++/WinRT takes advantage of this by producing C++ `noexcept` implementations of both the consuming and authoring code. If you have API methods or properties that are fail-free, and you're concerned about code size, then you can investigate this attribute.

Optimized code-generation

C++/WinRT now generates even more efficient C++ source code (behind the scenes) so that the C++ compiler can produce the smallest and most efficient binary code possible. Many of the improvements are geared toward reducing the cost of exception-handling by avoiding unnecessary unwind information. Binaries that use large amounts of C++/WinRT code will see roughly a 4% reduction in code size. The code is also more efficient (it runs faster) due to the reduced instruction count.

These improvements rely on a new interop feature that's available to you, as well. All of the C++/WinRT types that are resource owners now include a constructor for taking ownership directly, avoiding the previous two-step approach.

C++/WinRT

```
ABI::Windows::Foundation::IStringable* raw = ...

IStringable projected(raw, take_ownership_from_abi);

printf("%ls\n", projected.ToString().c_str());
```

Optimized exception-handling (EH) code-generation

This change complements work that has been done by the Microsoft C++ optimizer team to reduce the cost of exception-handling. If you use application binary interfaces (ABIs) (such as COM) heavily in your code, then you'll observe a lot of code following this pattern.

C++

```
int32_t Function() noexcept
{
    try
    {
        // code here constitutes unique value.
    }
    catch (...)
    {
        // code here is always duplicated.
    }
}
```

C++/WinRT itself generates this pattern for every API that's implemented. With thousands of API functions, any optimization here can be significant. In the past, the optimizer wouldn't detect that those catch blocks are all identical, so it was duplicating a lot of code around each ABI (which in turn contributed to the belief that using exceptions in system code produces large binaries). However, from Visual Studio 2019

on, the C++ compiler folds all of those catch funclets, and only stores those that are unique. The result is a further and overall 18% reduction in code size for binaries that rely heavily on this pattern. Not only is EH code now more efficient than using return codes, but also the concern about larger binaries is now a thing of the past.

Incremental build improvements

The `cppwinrt.exe` tool now compares the output of a generated header/source file against the contents of any existing file on disk, and it only writes out the file if the file has in fact changed. This saves considerable time with disk I/O, and it ensures that the files are not considered "dirty" by the C++ compiler. The result is that recompilation is avoided, or reduced, in many cases.

Generic interfaces are now all generated

Due to the xlang metadata reader, C++/WinRT now generates all parameterized, or generic, interfaces from metadata. Interfaces such as Windows::Foundation::Collections::IVector<T> are now generated from metadata rather than hand-written in `winrt/base.h`. The result is that the size of `winrt/base.h` has been cut in half, and that optimizations are generated right into the code (which was tricky to do with the hand-rolled approach).

> **ⓘ Important**
>
> Interfaces such as the example given now appear in their respective namespace headers, rather than in `winrt/base.h`. So, if you have not already done so, you'll have to include the appropriate namespace header in order to use the interface.

Component optimizations

This update adds support for several additional opt-in optimizations for C++/WinRT, described in the sections below. Because these optimizations are breaking changes (which you may need to make minor changes to support), you'll need to turn them on explicitly. In Visual Studio, set project property **Common Properties** > **C++/WinRT** > **Optimized** to *Yes*. That has the effect of adding `<CppWinRTOptimized>true</CppWinRTOptimized>` to your project file. And it has the same effect as adding the `-opt[imize]` switch when invoking `cppwinrt.exe` from the command line.

A new project (from a project template) will use `-opt` by default.

Uniform construction, and direct implementation access

These two optimizations allow your component direct access to its own implementation types, even when it's only using the projected types. There's no need to use **make**, **make_self**, nor **get_self** if you simply want to use the public API surface. Your calls will compile down to direct calls into the implementation, and those might even be entirely inlined.

For more info, and code examples, see Opt in to uniform construction, and direct implementation access.

Type-erased factories

This optimization avoids the #include dependencies in `module.g.cpp` so that it need not be recompiled every time any single implementation class happens to change. The result is improved build performance.

Smarter and more efficient `module.g.cpp` for large projects with multiple libs

The `module.g.cpp` file now also contains two additional composable helpers, named **winrt_can_unload_now**, and **winrt_get_activation_factory**. These have been designed for larger projects where a DLL is composed of a number of libs, each with its own runtime classes. In that situation, you need to manually stitch together the DLL's **DllGetActivationFactory** and **DllCanUnloadNow**. These helpers make it much easier for you to do that, by avoiding spurious origination errors. The `cppwinrt.exe` tool's `-lib` flag may also be used to give each individual lib its own preamble (rather than `winrt_xxx`) so that each lib's functions may be individually named, and thus combined unambiguously.

Coroutine support

Coroutine support is included automatically. Previously, the support resided in multiple places, which we felt was too limiting. And then temporarily for v2.0, a `winrt/coroutine.h` header file was necessary, but that's no longer needed. Since the Windows Runtime async interfaces are now generated, rather than hand-written, they now reside in `winrt/Windows.Foundation.h`. Apart from being more maintainable and supportable, it means that coroutine helpers such as **resume_foreground** no longer have to be tacked on to the end of a specific namespace header. Instead, they can more naturally include their dependencies. This further allows **resume_foreground** to support not only resuming on a given **Windows::UI::Core::CoreDispatcher**, but it can now also

support resuming on a given **Windows::System::DispatcherQueue**. Previously, only one could be supported; but not both, since the definition could only reside in one namespace.

Here's an example of the **DispatcherQueue** support.

C++/WinRT

```
...
#include <winrt/Windows.System.h>
using namespace Windows::System;
...
fire_and_forget Async(DispatcherQueueController controller)
{
    bool queued = co_await resume_foreground(controller.DispatcherQueue());
    assert(queued);

    // This is just to simulate queue failure...
    co_await controller.ShutdownQueueAsync();

    queued = co_await resume_foreground(controller.DispatcherQueue());
    assert(!queued);
}
```

The coroutine helpers are now also decorated with `[[nodiscard]]`, thereby improving their usability. If you forget to (or don't realize you have to) `co_await` them for them to work then, due to `[[nodiscard]]`, such mistakes now produce a compiler warning.

Help with diagnosing direct (stack) allocations

Since the projected and implementation class names are (by default) the same, and only differ by namespace, it's possible to mistake the one for the other, and to accidentally create an implementation on the stack, rather than using the **make** family of helpers. This can be hard to diagnose in some cases, because the object may be destroyed while outstanding references are still in flight. An assertion now picks this up, for debug builds. While the assertion doesn't detect stack allocation inside a coroutine, it's nevertheless helpful in catching most such mistakes.

For more info, see Diagnosing direct allocations.

Improved capture helpers, and variadic delegates

This update fixes the limitation with the capture helpers by supporting projected types as well. This comes up now and then with the Windows Runtime interop APIs, when they return a projected type.

This update also adds support for **get_strong** and **get_weak** when creating a variadic (non-Windows Runtime) delegate.

Support for deferred destruction and safe QI during destruction

It's not uncommon in the destructor of a runtime class object to call a method that temporarily bumps the reference count. When the reference count returns to zero, the object destructs a second time. In a XAML application, you might need to perform a **QueryInterface** (QI) in a destructor, in order to call some cleanup implementation up or down the hierarchy. But the object's reference count has already reached zero, so that QI constitutes a reference count bounce, too.

This update adds support for debouncing the reference count, ensuring that once it reaches zero it can never be resurrected; while still allowing you to QI for any temporary that you require during destruction. This procedure is unavoidable in certain XAML applications/controls, and C++/WinRT is now resilient to it.

You can defer destruction by providing a static **final_release** function on your implementation type. The last remaining pointer to the object, in the form of a **std::unique_ptr**, is passed to your **final_release**. You can then opt to move ownership of that pointer to some other context. It's safe for you to QI the pointer without triggering a double-destruction. But the net change to the reference count must be zero at the point you destruct the object.

The return value of **final_release** can be `void`, an asynchronous operation object such as **IAsyncAction**, or **winrt::fire_and_forget**.

C++/WinRT

```
struct Sample : implements<Sample, IStringable>
{
    hstring ToString()
    {
        return L"Sample";
    }

    ~Sample()
    {
        // Called when the unique_ptr below is reset.
    }

    static void final_release(std::unique_ptr<Sample> self) noexcept
    {
        // Move 'self' as needed to delay destruction.
    }
};
```

In the example below, once the **MainPage** is released (for the final time), **final_release** is called. That function spends five seconds waiting (on the thread pool), and then it resumes using the page's **Dispatcher** (which requires QI/AddRef/Release to work). It then cleans up a resource on that UI thread. And finally it clears the **unique_ptr**, which causes the **MainPage** destructor to actually get called. Even in that destructor, **DataContext** is called, which requires a QI for **IFrameworkElement**.

You don't have to implement your **final_release** as a coroutine. But that does work, and it makes it very simple to move destruction to a different thread, which is what's happening in this example.

C++/WinRT

```
struct MainPage : PageT<MainPage>
{
    MainPage()
    {
    }

    ~MainPage()
    {
        DataContext(nullptr);
    }

    static IAsyncAction final_release(std::unique_ptr<MainPage> self)
    {
        co_await 5s;

        co_await resume_foreground(self->Dispatcher());
        co_await self->resource.CloseAsync();

        // The object is destructed normally at the end of final_release,
        // when the std::unique_ptr<MyClass> destructs. If you want to
destruct
        // the object earlier than that, then you can set *self* to
`nullptr`.
        self = nullptr;
    }
};
```

For more info, see Deferred destruction.

Improved support for COM-style single interface inheritance

As well as for Windows Runtime programming, C++/WinRT is also used to author and consume COM-only APIs. This update makes it possible to implement a COM server where there exists an interface hierarchy. This isn't required for the Windows Runtime; but it is required for some COM implementations.

Correct handling of `out` params

It can be tricky to work with `out` params; particularly Windows Runtime arrays. With this update, C++/WinRT is considerably more robust and resilient to mistakes when it comes to `out` params and arrays; whether those parameters arrive via a language projection, or from a COM developer who's using the raw ABI, and who's making the mistake of not initializing variables consistently. In either case, C++/WinRT now does the right thing when it comes to handing off projected types to the ABI (by remembering to release any resources), and when it comes to zeroing out or clearing out parameters that arrive across the ABI.

Events now handle invalid tokens reliably

The **winrt::event** implementation now gracefully handles the case where its **remove** method is called with an invalid token value (a value that's not present in the array).

Coroutine local variables are now destroyed before the coroutine returns

The traditional way of implementing a coroutine type may allow local variables within the coroutine to be destroyed *after* the coroutine returns/completes (rather than prior to final suspension). The resumption of any waiter is now deferred until final suspension, in order to avoid this problem and to accrue other benefits.

News, and changes, in Windows SDK version 10.0.17763.0 (Windows 10, version 1809)

The table below contains news and changes for C++/WinRT in the Windows SDK version 10.0.17763.0 (Windows 10, version 1809).

Expand table

New or changed feature	More info
Breaking change. For it to compile, C++/WinRT doesn't depend on headers from the Windows SDK.	See Isolation from Windows SDK header files, below.
The Visual Studio project system format has changed.	See How to retarget your C++/WinRT project to a later version of the Windows SDK, below.

New or changed feature	More info
There are new functions and base classes to help you pass a collection object to a Windows Runtime function, or to implement your own collection properties and collection types.	See Collections with C++/WinRT.
You can use the {Binding} markup extension with your C++/WinRT runtime classes.	For more info, and code examples, see Data binding overview.
Support for canceling a coroutine allows you to register a cancellation callback.	For more info, and code examples, see Canceling an asynchronous operation, and cancellation callbacks.
When creating a delegate pointing to a member function, you can establish a strong or a weak reference to the current object (instead of a raw *this* pointer) at the point where the handler is registered.	For more info, and code examples, see the **If you use a member function as a delegate** sub-section in the section Safely accessing the *this* pointer with an event-handling delegate.
Bugs are fixed that were uncovered by Visual Studio's improved conformance to the C++ standard. The LLVM and Clang toolchain is also better leveraged to validate C++/WinRT's standards conformance.	You'll no longer encounter the issue described in Why won't my new project compile? I'm using Visual Studio 2017 (version 15.8.0 or later), and SDK version 17134

Other changes.

- **Breaking change**. **winrt::get_abi(winrt::hstring const&)** now returns `void*` instead of `HSTRING`. You can use `static_cast<HSTRING>(get_abi(my_hstring));` to get an HSTRING. See Interoperating with the ABI's HSTRING.
- **Breaking change**. **winrt::put_abi(winrt::hstring&)** now returns `void**` instead of `HSTRING*`. You can use `reinterpret_cast<HSTRING*>(put_abi(my_hstring));` to get an HSTRING*. See Interoperating with the ABI's HSTRING.
- **Breaking change**. HRESULT is now projected as **winrt::hresult**. If you need an HRESULT (to do type checking, or to support type traits), then you can `static_cast` a **winrt::hresult**. Otherwise, **winrt::hresult** converts to HRESULT, as long as you include `unknwn.h` before you include any C++/WinRT headers.
- **Breaking change**. GUID is now projected as **winrt::guid**. For APIs that you implement, you must use **winrt::guid** for GUID parameters. Otherwise, **winrt::guid** converts to GUID, as long as you include `unknwn.h` before you include any C++/WinRT headers. See Interoperating with the ABI's GUID struct.
- **Breaking change**. The **winrt::handle_type constructor** has been hardened by making it explicit (it's now harder to write incorrect code with it). If you need to assign a raw handle value, call the **handle_type::attach function** instead.

- **Breaking change**. The signatures of **WINRT_CanUnloadNow** and **WINRT_GetActivationFactory** have changed. You mustn't declare these functions at all. Instead, include `winrt/base.h` (which is automatically included if you include any C++/WinRT Windows namespace header files) to include the declarations of these functions.
- For the **winrt::clock struct**, **from_FILETIME/to_FILETIME** are deprecated in favor of **from_file_time/to_file_time**.
- Simplified APIs that expect **IBuffer** parameters. Most APIs prefer collections, or arrays. But we felt that we should make it easier to call APIs that rely on **IBuffer**. This update provides direct access to the data behind an **IBuffer** implementation. It uses the same data naming convention as the one used by the C++ Standard Library containers. That convention also avoids collisions with metadata names that conventionally begin with an uppercase letter.
- Improved code generation: various improvements to reduce code size, improve inlining, and optimize factory caching.
- Removed unnecessary recursion. When the command-line refers to a folder, rather than to a specific `.winmd`, the `cppwinrt.exe` tool no longer searches recursively for `.winmd` files. The `cppwinrt.exe` tool also now handles duplicates more intelligently, making it more resilient to user error, and to poorly-formed `.winmd` files.
- Hardened smart pointers. Formerly, the event revokers failed to revoke when move-assigned a new value. This helped uncover an issue where smart pointer classes weren't reliably handling self-assignment; rooted in the **winrt::com_ptr struct template**. **winrt::com_ptr** has been fixed, and the event revokers fixed to handle move semantics correctly so that they revoke upon assignment.

> **ⓘ Important**
>
> Important changes were made to the C++/WinRT Visual Studio Extension (VSIX), both in version 1.0.181002.2, and then later in version 1.0.190128.4. For details of these changes, and how they affect your existing projects, Visual Studio support for C++/WinRT and Earlier versions of the VSIX extension.

Isolation from Windows SDK header files

This is potentially a breaking change for your code.

For it to compile, C++/WinRT no longer depends on header files from the Windows SDK. Header files in the C run-time library (CRT) and the C++ Standard Template Library (STL) also don't include any Windows SDK headers. And that improves standards

compliance, avoids inadvertent dependencies, and greatly reduces the number of macros that you have to guard against.

This independence means that C++/WinRT is now more portable and standards compliant, and it furthers the possibility of it becoming a cross-compiler and cross-platform library. It also means that the C++/WinRT headers aren't adversely affected macros.

If you previously left it to C++/WinRT to include any Windows headers in your project, then you'll now need to include them yourself. It is, in any case, always best practice to explicitly include the headers that you depend on, and not leave it to another library to include them for you.

Currently, the only exceptions to Windows SDK header file isolation are for intrinsics, and numerics. There are no known issues with these last remaining dependencies.

In your project, you can re-enable interop with the Windows SDK headers if you need to. You might, for example, want to implement a COM interface (rooted in **IUnknown**). For that example, include `unknwn.h` before you include any C++/WinRT headers. Doing so causes the C++/WinRT base library to enable various hooks to support classic COM interfaces. For a code example, see Author COM components with C++/WinRT. Similarly, explicitly include any other Windows SDK headers that declare types and/or functions that you want to call.

How to retarget your C++/WinRT project to a later version of the Windows SDK

The method for retargeting your project that's likely to result in the fewest compiler and linker issue is also the most labor-intensive. That method involves creating a new project (targeting the Windows SDK version of your choice), and then copying files over to your new project from your old. There will be sections of your old `.vcxproj` and `.vcxproj.filters` files that you can just copy over to save you adding files in Visual Studio.

However, there are two other ways to retarget your project in Visual Studio.

- Go to project property **General** > **Windows SDK Version**, and select **All Configurations** and **All Platforms**. Set **Windows SDK Version** to the version that you want to target.
- In **Solution Explorer**, right-click the project node, click **Retarget Projects**, choose the version(s) you wish to target, and then click **OK**.

If you encounter any compiler or linker errors after using either of these two methods, then you can try cleaning the solution (**Build** > **Clean Solution** and/or manually delete all temporary folders and files) before trying to build again.

If the C++ compiler produces "*error C2039: 'IUnknown': is not a member of '`global namespace'*", then add `#include <unknwn.h>` to the top of your `pch.h` file (before you include any C++/WinRT headers).

You may also need to add `#include <hstring.h>` after that.

If the C++ linker produces "*error LNK2019: unresolved external symbol _WINRT_CanUnloadNow@0 referenced in function _VSDesignerCanUnloadNow@0*", then you can resolve that by adding `#define _VSDESIGNER_DONT_LOAD_AS_DLL` to your `pch.h` file.

Feedback

Frequently-asked questions about C++/WinRT

FAQ

Answers to questions that you're likely to have about authoring and consuming Windows Runtime APIs with C++/WinRT.

> **Important**
>
> For release notes about C++/WinRT, see News, and changes, in C++/WinRT 2.0.

> **Note**
>
> If your question is about an error message that you've seen, then also see the Troubleshooting C++/WinRT topic.

Where can I find C++/WinRT sample apps?

See C++/WinRT sample apps.

How do I retarget my C++/WinRT project to a later version of the Windows SDK?

See How to retarget your C++/WinRT project to a later version of the Windows SDK.

Why won't my new project compile, now that I've moved to C++/WinRT 2.0?

For the full set of changes (including breaking changes), see News, and changes, in C++/WinRT 2.0. For example, if you're using a range-based `for` on a Windows Runtime collection, then you'll now need to `#include <winrt/Windows.Foundation.Collections.h>`.

Why won't my new project compile? I'm using Visual Studio 2017 (version 15.8.0 or higher), and SDK version 17134

If you're using Visual Studio 2017 (version 15.8.0 or higher), and targeting the Windows SDK version 10.0.17134.0 (Windows 10, version 1803), then a newly created C++/WinRT project may fail to compile with the error "*error C3861: 'from_abi': identifier not found*", and with other errors originating in *base.h*. The solution is to either target a later (more conformant) version of the Windows SDK, or set project property **C/C++** > **Language** > **Conformance mode: No** (also, if **/permissive-** appears in project property **C/C++** > **Command Line** under **Additional Options**, then delete it).

How do I resolve the build error "The C++/WinRT VSIX no longer provides project build support. Please add a project reference to the Microsoft.Windows.CppWinRT Nuget package"?

Install the **Microsoft.Windows.CppWinRT** NuGet package into your project. For details, see Earlier versions of the VSIX extension.

How do I customize the build support in the NuGet package?

C++/WinRT build support (props/targets) is documented in the Microsoft.Windows.CppWinRT NuGet package readme.

What are the requirements for the C++/WinRT Visual Studio Extension (VSIX)?

For version 1.0.190128.4 of the VSIX extension and later, see Visual Studio support for C++/WinRT. For other versions, see Earlier versions of the VSIX extension.

What's a runtime class?

A runtime class is a type that can be activated and consumed via modern COM interfaces, typically across executable boundaries. However, a runtime class can also be used within the compilation unit that implements it. You declare a runtime class in Interface Definition Language (IDL), and you can implement it in standard C++ using C++/WinRT.

What do the projected type and the implementation type mean?

If you're only *consuming* a Windows Runtime class (runtime class), then you'll be dealing exclusively with *projected types*. C++/WinRT is a *language projection*, so projected types are part of the surface of the Windows Runtime that's *projected* into C++ with C++/WinRT. For more details, see Consume APIs with C++/WinRT.

The *implementation type* contains the implementation of a runtime class, so it's only available in the project that implements the runtime class. When you're working in a project that implements runtime classes (a Windows Runtime component project, or a project that uses XAML UI), it's important to be comfortable with the distinction between your implementation type for a runtime class, and the projected type that represents the runtime class projected into C++/WinRT. For more details, see Author APIs with C++/WinRT.

Do I need to declare a constructor in my runtime class's IDL?

Only if the runtime class is designed to be consumed from outside its implementing compilation unit (it's a Windows Runtime component intended for general consumption by Windows Runtime client apps). For full details on the purpose and consequences of declaring constructor(s) in IDL, see Runtime class constructors.

Why is the compiler giving me a "C3779: consume_Something: function

that returns 'auto' cannot be used before it is defined" error?

You're using a Windows Runtime object without having first included the corresponding namespace header file. Include the header named for the API's namespace, and rebuild. For more info, see C++/WinRT projection headers.

Why is the linker giving me a "LNK2019: Unresolved external symbol" error?

If the unresolved symbol is a Windows Runtime free function, such as RoInitialize, then you'll need to explicitly link the WindowsApp.lib umbrella library in your project. The C++/WinRT projection depends on some of these free (non-member) functions and entry points. If you use one of the C++/WinRT Visual Studio Extension (VSIX) project templates for your application, then `WindowsApp.lib` is linked for you automatically. Otherwise, you can use project link settings to include it, or do it in source code.

C++/WinRT

```
#pragma comment(lib, "windowsapp")
```

It's important that you resolve any linker errors that you can by linking **WindowsApp.lib** instead of an alternative static-link library, otherwise your application won't pass the Windows App Certification Kit tests used by Visual Studio and by the Microsoft Store to validate submissions (meaning that it consequently won't be possible for your application to be successfully ingested into the Microsoft Store).

If the unresolved symbol is a constructor, you may have forgotten to include the namespace header file for the class being constructed. Include the header named for the class's namespace, and rebuild. For more info, see C++/WinRT projection headers.

Why am I getting a "class not registered" exception?

In this case, the symptom is that—when constructing a runtime class or accessing a static member—you see an exception thrown at runtime with a HRESULT value of REGDB_E_CLASSNOTREGISTERED.

One cause can be that your Windows Runtime component can't be loaded. Make sure that the component's Windows Runtime metadata file (`.winmd`) has the same name as the component binary (the `.dll`), which is also the name of the project and the name of the root namespace. Also make sure that the Windows Runtime metadata and the binary have been corectly copied by the build process to the consuming app's `Appx` folder. And confirm that the consuming app's `AppxManifest.xml` (also in the `Appx` folder) contains an <**InProcessServer**> element correctly declaring the activatable class and the binary name.

Uniform construction This error can also happen if you try to instantiate a locally-implemented runtime class via any of the projected type's constructors (other than its **std::nullptr_t** constructor). To do that, you'll need the C++/WinRT 2.0 feature that's often called uniform construction. If you want to opt in to that feature, then for more info, and code examples, see Opt in to uniform construction, and direct implementation access.

For a way of instantiating your locally-implemented runtime classes that *doesn't* require uniform construction, see XAML controls; bind to a C++/WinRT property.

Should I implement Windows::Foundation::IClosable and, if so, how?

If you have a runtime class that frees resources in its destructor, and that runtime class is designed to be consumed from outside its implementing compilation unit (it's a Windows Runtime component intended for general consumption by Windows Runtime client apps), then we recommend that you also implement **IClosable** in order to support the consumption of your runtime class by languages that lack deterministic finalization. Make sure that your resources are freed whether the destructor, **IClosable::Close**, or both are called. **IClosable::Close** may be called an arbitrary number of times.

Do I need to call IClosable::Close on runtime classes that I consume?

IClosable exists to support languages that lack deterministic finalization. So, in general, you don't need to call **IClosable::Close** from C++/WinRT. But consider these exceptions to that general rule.

- There are very rare cases involving shutdown races or semi-deadly embraces, where you do need to call **IClosable::Close**. If you're using

Windows.UI.Composition types, as an example, then you may encounter cases where you want to dispose objects in a set sequence, as an alternative to allowing the destruction of the C++/WinRT wrapper do the work for you.

- If you can't guarantee that you have the last remaining reference to an object (because you passed it to other APIs, which could be keeping a reference), then calling **IClosable::Close** is a good idea.
- When in doubt, it's safe to call **IClosable::Close** manually, rather than waiting for the wrapper to call it on destruction.

So, if you know that you have the last reference, then you can let the wrapper destructor do the work. If you need to close before the last reference vanishes, then you need to call **Close**. To be exception-safe, you should **Close** in a resource-acquisition-is-initialization (RAII) type (so that close happens on unwind). C++/WinRT doesn't have a **unique_close** wrapper, but you can make your own.

Can I use LLVM/Clang to compile with C++/WinRT?

We don't support the LLVM and Clang toolchain for C++/WinRT, but we do make use of it internally to validate C++/WinRT's standards conformance. For example, if you wanted to emulate what we do internally, then you could try an experiment such as the one described below.

Go to the LLVM Download Page, look for **Download LLVM 6.0.0** > **Pre-Built Binaries**, and download **Clang for Windows (64-bit)**. During installation, opt to add LLVM to the PATH system variable so that you'll be able to invoke it from a command prompt. For the purposes of this experiment, you can ignore any "Failed to find MSBuild toolsets directory" and/or "MSVC integration install failed" errors, if you see them. There are a variety of ways to invoke LLVM/Clang; the example below shows just one way.

Windows Command Prompt

```
C:\ExperimentWithLLVMClang>type main.cpp
// main.cpp
#pragma comment(lib, "windowsapp")
#pragma comment(lib, "ole32")

#include <winrt/Windows.Foundation.h>
#include <stdio.h>
#include <iostream>

using namespace winrt;

int main()
```

```
{
    winrt::init_apartment();
    Windows::Foundation::Uri rssFeedUri{ L"https://blogs.windows.com/feed"
};
    std::wcout << rssFeedUri.Domain().c_str() << std::endl;
}

C:\ExperimentWithLLVMClang>clang-cl main.cpp /EHsc /I ..\.. -Xclang -
std=c++17 -Xclang -Wno-delete-non-virtual-dtor -o app.exe

C:\ExperimentWithLLVMClang>app
windows.com
```

Because C++/WinRT uses features from the C++17 standard, you'll need to use whatever compiler flags are necessary to get that support; such flags differ from one compiler to another.

Visual Studio is the development tool that we support and recommend for C++/WinRT. See Visual Studio support for C++/WinRT.

Why doesn't the generated implementation function for a read-only property have the const qualifier?

When you declare a read-only property in MIDL 3.0, you might expect the `cppwinrt.exe` tool to generate an implementation function for you that is `const`-qualified (a const function treats the *this* pointer as const).

We certainly recommend using const wherever possible, but the `cppwinrt.exe` tool itself doesn't attempt to reason about which implementation functions might conceivably be const, and which might not. You can choose to make any of your implementation functions const, as in this example.

C++/WinRT

```
struct MyStringable : winrt::implements<MyStringable,
winrt::Windows::Foundation::IStringable>
{
    winrt::hstring ToString() const
    {
        return L"MyStringable";
    }
};
```